MNXB11: Homework 6 Assignment

Matei Popescu

October 2026

1 Creating ROOT Classes

Question 0

In your answers, explain what these mechanisms prevent and how the two approaches differ?

Header guards introduce a manual check that forces the compiler to run the `#ifndef` directive prior to copying the code into the relevant translation unit at compile time; if it has not been utilised before, the check passes, and the headers and relevant content get translated in preparation for linking. `#pragma once` instead instructs the compiler to remember if the header file has been accessed previously at compile time. For *most* projects, I think these should be largely equivalent. The `#pragma once` approach might run into issues if the compiler can't properly traverse the directory structure of the project, but I think that's only a concern if you are doing some really dubious stuff. The header guard approach can also fail if there are too many headers and you reuse guard names. As an aside, I believe for portability the header guard approach is technically superior by a slim margin since all C++ implementations support it but not necessarily the other.

Question 1

1. (0.05 points) Explain the difference between the default access rules of struct and class.

A `struct` has public members and public inheritance by default unless otherwise excluded with `protected:`. A `class` has private members and private inheritance by default unless overwritten with a `public:` block. Both support the same method implementation and features.

Question 2

2. (0.05 points) Why do we need our class to be derived from the TObject?

Because this way ROOT allows us to run our code with its pre-written class methods and special interfaces, such as utilising ROOT Collections, utilising `Draw()` and so on. This way ROOT does not need to even know what specific type the object is.

Question 3

3. (0.20 points) Implement the class interface and verify that the class compiles without errors.

Implemented and verified!

2 Creating the Ttree

Question 4

(0.10 points) Add the three velocity components to your class and initialize them correctly in both constructors. Verify that the getters and SetVelocity work

Everything works just fine.

Question 5

(0.15 points) Implement `Speed()` and `HorizontalSpeed()`. Check your functions using the following examples:

Example	v_x (m/s)	v_y (m/s)	v_z (m/s)
A	0.30	0.40	0.00
B	0.30	0.40	0.50

Report both speeds for each example in your answers. Explain why changing v_z changes the total speed but leaves the horizontal speed unchanged. Check that $0 \leqslant v_{xy} \leqslant v$.

For A, the speed is $\|\mathbf{v}\| = 0.5$ and the horizontal speed is $\|\mathbf{v}_{xy}\| = 0.5$.
For B, the speed is $\|\mathbf{v}\| = 0.70716078$ and the horizontal speed is still $\|\mathbf{v}_{xy}\| = 0.5$.
This makes sense; the formula for `.HorizontalSpeed()` just does not contain the z-component, unlike the one for `.Speed()`. And for good reason, if you compute the velocity on a given plane, you're intentionally neglecting movement outside said plane.

Question

(0.20 points) Generate and store 10,000 measurements in `tree_file.root`, using the tree name `tree` and branch name `object`. Verify the entry count. Include the random seed and generation parameters in your answers.

Respectively, done, verified. The random seed is 12345, and generation parameters are a standard deviation of 0.30 m/s and a mean of 0.0 m/s. All of these apply to a Gaussian distribution; each draw is fresh.

Question 7

(0.25 points) Fill and draw the two-dimensional histogram of v_y versus v_x. Save it as `velocity_components.pdf` or `velocity_components.png`. Give it a descriptive title and label both axes with units.

Done! Plots match expectations, I think.
Commands necessary are just `write()` and then `read()`.